

PySPEDAS Tutorial

Jim Lewis

jwl@ssl.Berkeley.edu

Presentation slides and notebooks:

https://github.com/spedas/pyspedas_examples/tree/master/pyspedas_examples/notebooks/agu_tutorial_2025

Agenda

- Getting started with Python, PySPEDAS, and related tools
- Basic PySPEDAS concepts
- PySPEDAS Load Routines (Mission-specific plugins)
- Other ways of loading data
 - HAPI
 - CDAWeb
 - MTH5 (Magnetotelluric data sets)
 - Importing directly from data files
- PySPEDAS Plot Routines
- Working with tplot variables

Agenda (continued)

- Coordinate transforms
- Working with 3-D Particle Distributions
- FFT/Wavelet/Power Spectrum tools

Getting started with Python and PySPEDAS

- Python requirements and installation
 - Anaconda recommended, but python.org, Windows store, etc will work. Download here: <https://www.anaconda.com/download>
 - Python 3.10 and later are supported for PySPEDAS; Python 3.14 is mostly supported, but some optional features are not yet compatible with Python 3.14
- Python interactive development environments
 - Optional, but recommended for convenience
 - This demo will use PyCharm (free version available), but many people use VS Code
 - You can also develop PySPEDAS scripts using Jupyter notebooks
- Installing PyCharm
 - Download here: <https://www.jetbrains.com/pycharm/download/>
 - Where to find key PyCharm features: see https://pyspedas.readthedocs.io/en/latest/getting_started.html#create-a-python-project-in-pycharm

Getting started (continued)

- Setting up a Python virtual environment for PySPEDAS
 - Your IDE probably offers a choice between ‘conda’ and ‘venv’ environments – we recommend conda.
 - Creating a new environment in Anaconda: see https://pyspedas.readthedocs.io/en/latest/getting_started.html#installing-python
 - Creating a PyCharm project to work with this environment: see https://pyspedas.readthedocs.io/en/latest/getting_started.html#create-a-python-project-in-pycharm
- Installing PySPEDAS
 - Install just the core dependencies, or optional extras as well?
 - Optional features are ‘maps’ (used by SECS/EICS plugin to plot magnetometer data over a map of North America), ‘vires’ (access to ESA’s VIRES archive service, including SWARM and some other missions), and ‘mth5’ (access to USGS magnetotelluric data from seismometer networks)
 - Some packages may be prone to dependency conflicts, or otherwise difficult to install on certain platforms (for example, older Macs)
 - In the terminal window (not the Python window!):

```
pip install 'pyspedas[all]' # installs everything
```
 - Or:

```
pip install pyspedas # just install core dependencies
```

Getting started (continued)

- Installing other useful tools
 - Most users will want at least jupyter, to work with notebooks.
In terminal window (not Python window!):
`pip install jupyter`
- Verifying that PySPEDAS and dependencies are successfully installed
 - Open an interactive Python window (not the terminal window!), then at the Python prompt:
`import pyspedas`
 - If there are no import errors, pyspedas should be good to go!

Working with PySPEDAS Jupyter notebooks

- Many PySPEDAS examples from today's tutorial will be presented as Jupyter notebooks. There are several repositories on GitHub with PySPEDAS examples:
 - General PySPEDAS features: https://github.com/spedas/pyspedas_examples
 - THEMIS examples: <https://github.com/spedas/themis-examples>
 - MMS examples: <https://github.com/spedas/mms-examples>
- You can download a notebook from Github and run it locally, in your own PySPEDAS installation. Navigate to a specific notebook you're interested in, then find the link to 'download raw file'. For example: https://github.com/spedas/pyspedas_examples/blob/master/pyspedas_examples/notebooks/agu_tutorial_2025/pyspedas_basics.ipynb

Download raw file link

Files

master + 🔍

Go to file t

- > .github
- ✓ pypedas_examples
 - > examples
 - ✓ notebooks
 - ✓ agu_tutorial_2025
 - pypedas_basics.ipynb
 - > mth5
 - Auroral Imagery with asilib.ipynb
 - BARREL_background_model.ip...
 - BARREL_basic usage.ipynb

pyypedas_examples / pypedas_examples / notebooks / agu_tutorial_2025 / pypedas_basics.ipynb

jameswilburlewis New notebook for AGU tutorial

9c2a1ca · 9 minutes ago History

Preview Code Blame 447 lines (447 loc) · 145 KB



The following line is useful when running PySPEDAS notebooks in Google Colab, to ensure that the pypedas package is installed in the environment.

You can comment this out, or skip this cell if you're running the notebook in an environment that already includes PySPEDAS.

In [12]: `!pip install pypedas`

```
16-Dec-25 21:03:42: /Users/jwl/anaconda3/envs/pycharm_py312/lib/python3.12/pty.py:95: DeprecationWarning: This process (pid=76781) is multi-threaded, use of forkpty() may lead to deadlocks in the child.
pid, fd = os.forkpty()
```

```
32.19s - pydevd: Sending message related to process being replaced timed-out after 5 seconds
Requirement already satisfied: pypedas in /Users/jwl/anaconda3/envs/pycharm_py312/lib/python3.12/site-packages (2.0.5)
```


Working with Jupyter notebooks

- After downloading, copy the notebook file into your PySPEDAS project (or anywhere you'll be able to find it later).
- Then, in your terminal window (not a Python window!):
`jupyter notebook /path/to/your/notebook`
- This should open a browser window with the notebook displayed and ready to run.

Another Jupyter option: Google Colab

- Google Colab is a service that lets users run Jupyter notebooks in their cloud environment – no local Python installation needed!
- You probably need a Google account to be able to use it.
- There is an interface to run a notebook directly from a Github repository. For example, go to <https://colab.research.google.com/>
- From that page, you should be able to select “GitHub” from the left hand pane, then enter the Github organization ‘spedas’ in the search box. This will bring up a list of repositories associated with SPEDAS.

Open notebook

Examples >

Recent >

Google Drive >

GitHub >

Upload >

+ New notebook

Enter a GitHub URL or search by organization or user

spedas



☐ Include private repos

Repository:



View repositories on GitHub

Branch:



spedas/bleeding_edge

master

spedas/bleeding_edge

spedas/cdflib

spedas/geopack

spedas/heliophysicsPy.github.io

spedas/mms-examples

spedas/mth5

spedas/pdm

spedas/pyspedas

spedas/pyspedas-add-project

spedas/pyspedas-release-scripts

spedas/pyspedas-validation

spedas/pyspedas_examples

spedas/pyspedas_gui

spedas/summer-school-24

Tables/make_csv/colormap_export.ipynb



Cancel

Google Colab (continued)

- Scroll through the list of SPEDAS repositories, find the one named 'pyspedas_examples', and click on it.
- The next screen will show a long list of available notebooks from that repository. The ones we'll be demo-ing today are in the folder notebooks/agu_tutorial_2025. Scroll through the list to find the pyspedas_basics.ipynb entry from that folder and click on it – that should bring up an interactive Jupyter session with that notebook loaded.



pyspedas_basics.ipynb

File Edit View Insert Runtime Tools Help



Share



Commands + Code + Text Run all Copy to Drive

Connect



The following line is useful when running PySPEDAS notebooks in Google Colab, to ensure that the pyspedas package is installed in the environment.

You can comment this out, or skip this cell if you're running the notebook in an environment that already includes PySPEDAS.

```
[ ] !pip install pyspedas

16-Dec-25 21:03:42: /Users/jwl/anaconda3/envs/pycharm_py312/lib/python3.12/pty.py:95: DeprecationWarning: This process (pid=76781) is multi-threaded, us
pid, fd = os.forkpty()

32.19s - pydevd: Sending message related to process being replaced timed-out after 5 seconds
Requirement already satisfied: pyspedas in /Users/jwl/anaconda3/envs/pycharm_py312/lib/python3.12/site-packages (2.0.5)
Requirement already satisfied: numpy>=1.20.0 in /Users/jwl/anaconda3/envs/pycharm_py312/lib/python3.12/site-packages (from pyspedas) (2.2.6)
Requirement already satisfied: requests>=2.31.0 in /Users/jwl/anaconda3/envs/pycharm_py312/lib/python3.12/site-packages (from pyspedas) (2.32.5)
Requirement already satisfied: geopack>=1.0.12 in /Users/jwl/anaconda3/envs/pycharm_py312/lib/python3.12/site-packages (from pyspedas) (1.0.12)
Requirement already satisfied: scipy>=1.11.1 in /Users/jwl/anaconda3/envs/pycharm_py312/lib/python3.12/site-packages (from pyspedas) (1.16.3)
Requirement already satisfied: cdflib>=1.0.0 in /Users/iwl/anaconda3/envs/bvcharm_bv312/lib/bvthon3.12/site-packages (from pyspedas) (1.3.6)
```

PySPEDAS features and basic usage

Basic PySPEDAS concepts

- Tplot variables
- Time
- Metadata
- See `pyspedas_basics.ipynb` notebook from Github
`pyspedas_examples` repo, under `notebooks/agu_tutorial_2025`

Working with tplot variables

- Converting Python arrays to tplot variables and vice versa
- Metadata in tplot variables
- Sanitizing data (dealing with NaNs, data gaps, spikes)
- Interpolating data
- Arithmetic operations
- Splitting/joining vector components
- See documentation:
<https://pyspedas.readthedocs.io/en/latest/utilities.html>
- See notebook: tplot_utilities.ipynb

PySPEDAS load routines

- Data caching
 - The environment variable SPEDAS_DATA_DIR, if defined, sets the root directory for downloaded data files. It can be shared between PySPEDAS and IDL SPEDAS installations
 - By default, PySPEDAS load routines request file modification times from the server before downloading. If there are cached files matching the request, and the server timestamps are older than the filesystem timestamps, the download is skipped and data is loaded from the cached files.
 - Mission-specific download URLs are defined in a config.py file in each mission directory. That mission's load routines will use that information internally to construct URLs for data requests.

PySPEDAS load routines (continued)

- Importing load routines
 - Each mission directly supported by PySPEDAS has a Python module within the `pyspedas.projects` namespace, where all that missions load routines and other tools can be found. Examples:

```
from pyspedas.projects.themis import fgm # Load routine for THEMIS FGM data
from pyspedas.projects.mms import feeps # Load routine for MMS FEEPS data
```

- Load routines can also be called using fully qualified names. This can be useful when working with multiple missions:

```
# THEMIS and MMS both have an fgm() load routine
mms_vars = pyspedas.projects.mms.fgm(probe='1',trange=trange)
thm_vars = pyspedas.projects.themis.fgm(probe='a',trange=trange)
```

PySPEDAS load routines (continued)

- Load routine parameters
 - We try to use consistent parameter names, where possible, between load routines. For example:
 - `probe`: A string specifying which probe of a multi-spacecraft mission should be loaded
 - `trange`: A two-element list or array containing start and end times (can be specified as Unix times or strings)
 - `level`: A string specifying the data level to load (e.g. 'l1', 'l1a', 'l2', 'ql')
 - `datatype`: A string (possibly including wildcards) indicating which datatypes should be loaded for this operation
 - `get_support_data`: A Boolean flag specifying whether to load ancillary data quantities marked as 'support_data' in the CDF metadata
 - Additional parameters may also be available (for example, 'prefix' or 'suffix' to modify tplot variable names).

PySPEDAS load routines (continued)

- Load routine documentation
 - Each mission has a dedicated page on pyspedas.readthedocs.io, with an overview of available instruments or data types for that mission, parameters accepted by the load routines, and often some simple examples of loading and plotting data for that mission. Example:
<https://pyspedas.readthedocs.io/en/latest/erg.html>
 - Python's 'help' routine can show documentation strings for a given load routine name, for example (in an interactive Python session):

```
help(pyspedas.projects.themis.fgm)
```
 - The `pyspedas.libs()` routine will show PySPEDAS routines whose names or descriptions match a provided string or wildcard pattern:

```
pyspedas.libs('fgm')
```
 - See notebook `pyspedas_load_routines.ipynb`
 - See documentation: <https://pyspedas.readthedocs.io/en/latest/projects.html>

Other data access methods

- HAPI (Heliophysics Application Program Interface)
- CDAWeb (SPDF's Common Data Analysis Web)
- VIRES (ESA service for accessing SWARM data)
- MTH5 (Magnetotelluric data from USGS seismometer sites)
- Importing directly from data files
 - CDF: `pyspedas.cdf_to_tplot`
 - NetCDF: `pyspedas.netcdf_to_tplot`
 - IDL sav files: `scipy.io.readsav`
 - IDL SPEDAS tplot save files: `pyspedas.tplot_restore`
- See notebook: `PySPEDAS_HAPI_CDAWeb.ipynb`

Plotting data

- Line plots
- Spectrograms
- Combining variables ('pseudovariables')
- Global plot options
- Per-variable plot options
- Interacting with plots
 - Zooming/panning
 - Interactive time selection (See standalone program `interactive_time_selection.py`)
- Saving plots to files
- Direct plot manipulation with Matplotlib
- See documentation: <https://pyspedas.readthedocs.io/en/latest/plotting.html>
- See notebooks:

`pyspedas_plot_options.ipynb`

`pyspedas_pseudo_variables.ipynb`

Coordinate transforms

- How coordinate system metadata is stored
- Well-known geophysical systems (GEl, GSE, GSM, J2000, SM, GEO)
- Mission-specific coordinate systems
 - Spinning/despun systems
 - RTN
 - ENP
- Special coordinate systems
 - Field-aligned coordinates
 - Minimum Variance Analysis
 - Boundary normal coordinates
- See notebooks:

Coordinate_transformations.ipynb

THEMIS_fac_matrix_make.ipynb

THEMIS_minimum_variance_analysis.ipynb

Working with 3-D Particle Distributions

- Basics of 3-d particle distributions
- Spectrograms
- Moments
- 2-D slices
- 1-D slices
- See notebooks:

Generate spectrograms and moments with `mms_part_getspec.ipynb`

`Generate_2D_slices_of_FPI_and_HPCA_data.ipynb`

Spectral methods (FFT, wavelets, power spectra)

- FFTs
- Wavelets
- Power spectra
- Dynamic power spectra
- See documentation:

<https://pyspedas.readthedocs.io/en/latest/utilities.html#wave-polarization-power-spectrum-operations>

- See notebooks:

MMS_Search-coil Magnetometer (SCM)_tdpwrspec.ipynb

Wave_polarization_using_MMS_SCM_data.ipynb

Magnetic Field Modeling

- Field models available
 - IGRF
 - T89
 - T96
 - TS01
 - TS04
 - Not yet implemented: TA15a, TA15n, TA16, TS07
- Model parameters
- Coordinate systems
- Time+position to model B vectors
- Work in progress: field line tracing